RIGHTALLY | ARCHITECTURE SPECIFICATION

RightAlly Platform Architecture

The all-in-one operating system for US real estate brokerages
Built for 100,000 brokerages and millions of agents

VERSION	1.0 - draft for review
DATE	November 7, 2026
PHASE	Phase 0 / Phase 2 input - architecture baseline
PREPARED BY	Mayura Consultancy Services, engineering
CLASSIFICATION	Confidential - RightAlly internal

Contents

1. Executive summary
2. Product vision and goals
3. Architecture principles
4. Key decisions and why
5. System architecture
6. Multi-tenancy and isolation
7. Scalability and performance
8. Technology stack
9. Module catalogue: what we build
10. Building a new module (Module SDK)
11. Data architecture
12. MLS and IDX architecture
13. AI architecture and guardrails
14. Releasing to all brokerages
15. Security and compliance
16. Reliability, observability and disaster recovery
17. Roadmap and team
18. Risks and mitigations
19. Decisions needed and assumptions

How to read this document. Sections 1-4 explain what we are building and why this architecture. Sections 5-8 show how it works and scales. Section 9 lists every module and its features. Sections 10 and 14 answer two specific questions: how we add new modules (and new databases) safely, and how every release reaches every brokerage. Section 19 lists the decisions we need from you to start Phase 1.

1. Executive summary

RightAlly will be a single, cloud-native SaaS platform that gives a US brokerage everything it runs on today across eight to twelve separate tools: IDX website, CRM, marketing automation, agent onboarding, forms and e-signature, transaction management, commission and revenue-share accounting, payouts, compliance and an AI assistant. Brokerages sign up, connect their MLS, choose a website template, point their domain and invite agents without calling us.

The architecture is a **modular monolith deployed in cells**: one Laravel 13 codebase made of independent modules, run as many identical, isolated cells behind a global control plane. Each cell serves up to about 2,000 brokerages or 50,000 agents. Growth is handled by adding cells, not by rewriting software. This is the same pattern used by large multi-tenant SaaS companies because it keeps development simple while making scale and failure isolation a matter of operations.

Question	Answer in this architecture
Can it reach 100,000 brokerages and millions of agents?	Yes. Capacity grows linearly with cells (about 40-80 cells at target), each independently scaled. MLS data and search are shared per MLS, so listing volume does not multiply per brokerage.
Will it be fast?	Public pages are cached at the edge (target under 100 ms worldwide). Logged-in API calls target p95 under 200 ms; listing search under 300 ms.
Is each brokerage's data isolated?	Yes, at five layers: routing, query scoping, storage prefixes and encryption keys, search filters and AI retrieval filters, verified by automated cross-tenant tests on every build.
Can we add new modules later?	Yes. Every feature is a module with a manifest, its own tables and a contract with Core. New modules can be sold as add-ons and switched on per brokerage and per agent.
How do updates reach brokerages?	Centrally. Every release goes through CI, a canary cell, then waves of cells with automatic rollback. Feature flags let us release new features to chosen brokerages first.
What does it cost to run?	Shared infrastructure per cell keeps unit cost low; a brokerage can still buy a dedicated cell for isolation.

Recommendation. Approve this architecture as the baseline for Phase 1 (product requirements) and Phase 2 (detailed design). It supersedes the earlier assumption in specs 02-04 of a separate deployment and database per customer, which does not scale beyond a few hundred brokerages.

2. Product vision and goals

A broker evaluating RightAlly should conclude: *everything we use today, in one place, and each part is the best available.* Agents should prefer it to the tools they bring from other brokerages. That sets three bars: depth in every module, one consistent experience across modules, and data that flows between modules without re-entry.

Business goals

Goal	Target	Architectural consequence
Brokerages on platform	100,000	Cell-based multi-tenancy, self-service onboarding, automated provisioning
Agents on platform	2-4 million registered	Horizontal scale, seat-based entitlements, mobile-first UX
Self-service	Broker live in under one day	Wizard: plan, MLS, template, domain, comp plan, invite agents
Module economy	New modules sold as add-ons	Module SDK, entitlements, marketplace, per-agent seats
Performance	Pages feel instant	Edge caching, Octane, read replicas, search engine for listings
Trust	Brokerage-grade compliance	Audit trail, MLS rule engine, e-sign evidence, SOC 2 Type II

Who uses it

Persona	Main jobs
Super admin (RightAlly)	Create packages and add-ons, onboard and support brokerages, release management, platform analytics, law and template libraries
Broker / owner	Website and brand, recruiting, comp plans, compliance review, finances and payouts, reports
Office manager / staff	Transaction review, document checklists, agent billing, onboarding approvals
Agent	Leads and CRM, landing pages, marketing, forms and e-sign, transactions, earnings, AI help
Consumer	Search homes on IDX sites, save searches, contact agents, sign documents

3. Architecture principles

Ten principles guide every decision in this document and in later phases. Where a later design conflicts with one, the conflict is written up as an Architecture Decision Record (ADR) and approved before code.

#	Principle	What it means in practice
1	One codebase, many cells	No customer forks. Customer differences are configuration, templates, workflows and modules, never code edits.
2	Modules with hard boundaries	Each module owns its tables and talks to others only through interfaces and events.
3	Tenant isolation by default	Code cannot query without a tenant context; tests prove it on every build.
4	API-first	Every screen uses the same versioned API the Flutter apps and partners will use.
5	Event-driven between modules	"Transaction closed" is an event; Finance, CRM, Marketing and Reports react independently.
6	Money is a ledger	Commissions, fees and payouts are immutable double-entry records; corrections are reversals.
7	Configurable, not custom	Comp plans, workflows, checklists, forms and templates are data the broker edits.
8	Secure and compliant by default	Encryption, least privilege, audit logs, MLS display rules and fair housing checks are built in, not optional.
9	Observable	Every request, job and release is traced and measured per cell and per brokerage.
10	Evolve without rewrites	Start as a modular monolith; extract a module to a service only when data proves it is needed.

4. Key decisions and why

Each decision below compares the realistic options and states why we chose one. These become the first ADRs.

4.1 Application structure

Option	Strengths	Weaknesses	Verdict
Classic monolith (today's code)	Simple to start	No boundaries; every change risks everything; cannot scale teams	Rejected
Microservices from day one	Independent scaling and deploys	10x operational cost, distributed transactions for money flows, slow delivery with a small team	Rejected for now
Modular monolith	One deploy, strong boundaries, fast delivery, transactions where money needs them	Needs discipline (enforced by architecture tests)	**Chosen**; modules can be extracted later

4.2 Multi-tenancy model

Option	Strengths	Weaknesses	Verdict
Deployment + DB per brokerage (specs 02-04, today)	Strong isolation	100k deployments to patch, migrate and back up; cost per tenant too high; no cross-brokerage analytics	Rejected
DB per brokerage, shared app	Good isolation	Every release migrates 100k databases; connection limits; schema drift	Rejected
One global shared database	Cheapest, simplest	Single point of failure and a hard scaling ceiling	Rejected
Cells with shared schema	Linear scale, blast radius limited to one cell, simple releases, dedicated cell option	Requires strict tenant scoping (designed in, Section 6)	**Chosen**

4.3 Other foundational choices

Area	Choice	Why
Backend	Laravel 13, PHP 8.5, Octane	Latest framework (security fixes to Q1 2028), team expertise, Octane keeps the app in memory for low latency
Front end	React 19 + TypeScript, Inertia 2 with SSR	Your preferred stack; SSR for SEO on public pages; one component library across portals
Primary database	MySQL 8.4 LTS on Amazon Aurora	Proven at scale, read replicas, point-in-time recovery, team expertise
Search	OpenSearch	Geo search and facets over tens of millions of listings; CRM full-text search
Cache, queue, realtime	Redis, Horizon, Reverb	First-party Laravel tooling, simple operations
Edge	Cloudflare (CDN, WAF, Cloudflare for SaaS)	Custom domains via CNAME with automatic SSL for thousands of brokerage domains
Cloud	AWS (recommended)	Managed Aurora, OpenSearch, EKS/ECS, KMS, multi-region; replaces today's single DigitalOcean host
Mobile	Flutter (later) on the same API	One API for web, mobile and partners

5. System architecture

Four layers: the edge, the global control plane, the cells where brokerage work happens, and shared platform services that are expensive to duplicate (MLS data, listing search, AI, analytics, messaging).

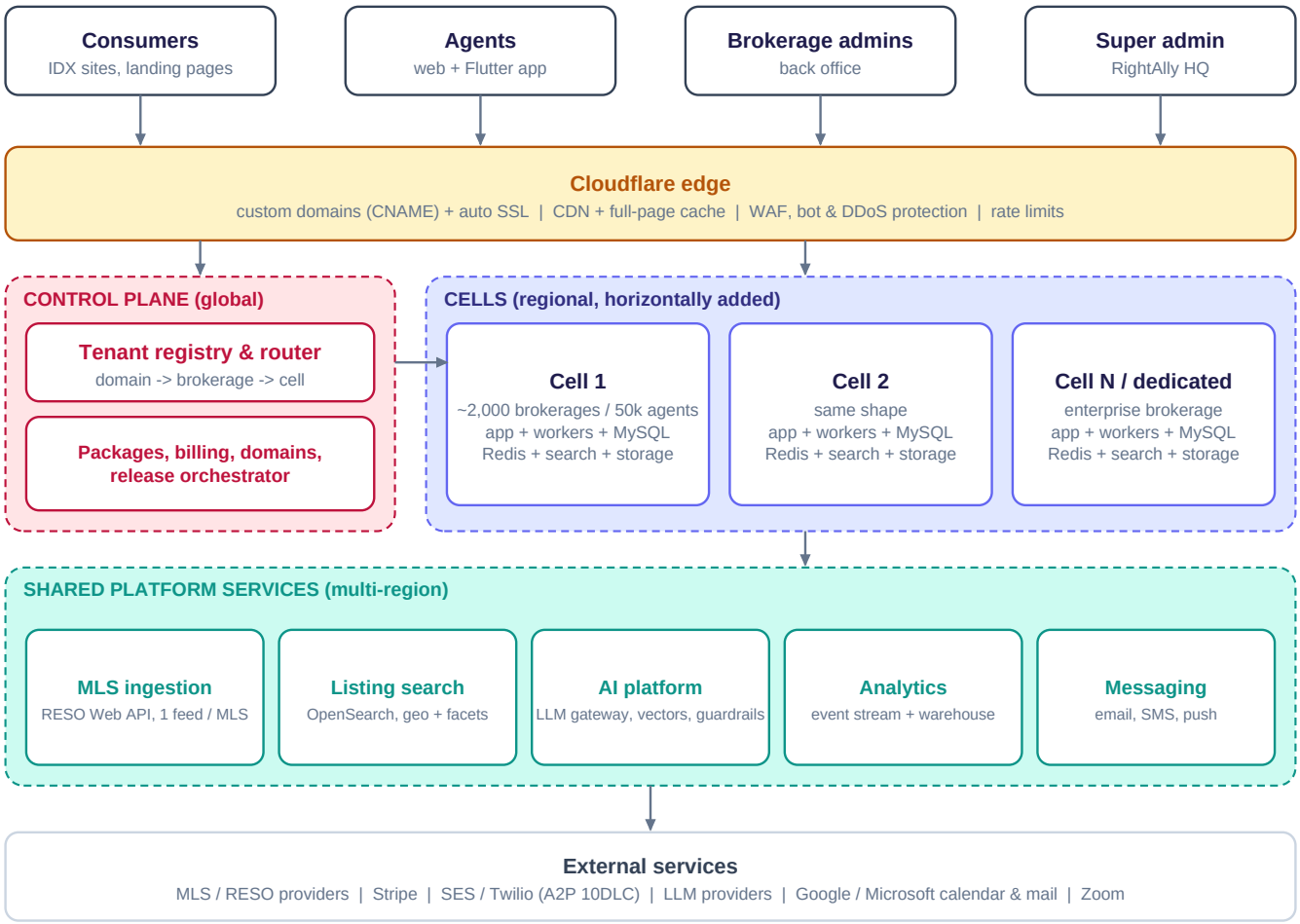


Figure 1 - System context: edge, control plane, cells and shared services

Layer	Responsibility
Edge	Terminates TLS for every brokerage domain, caches public pages and assets, blocks attacks, enforces rate limits.
Control plane	Global registry of brokerages, domains, packages, subscriptions and which cell each brokerage lives in. Provisions new brokerages in seconds. Orchestrates releases.
Cells	Run the full application for their brokerages. Identical software; size and region vary.
Shared services	MLS ingestion and listing search are per MLS, not per brokerage. AI, analytics and messaging are shared with tenant tagging.

5.1 Inside a cell

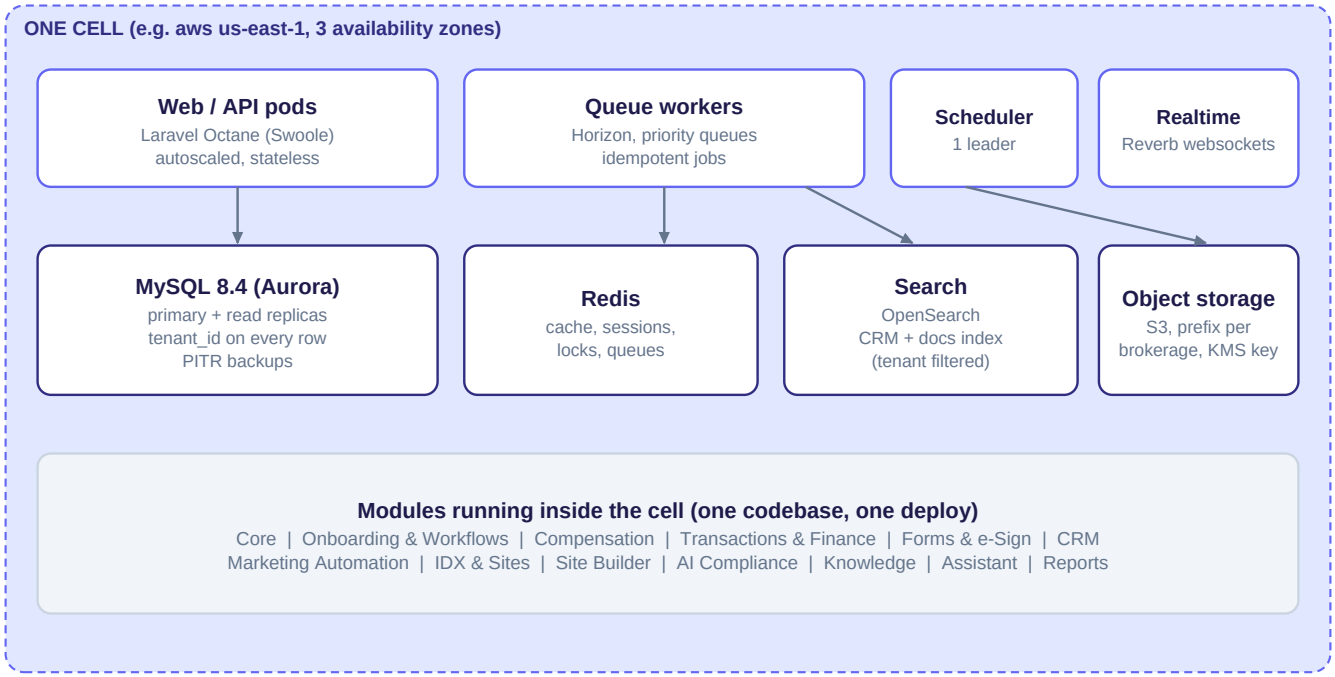
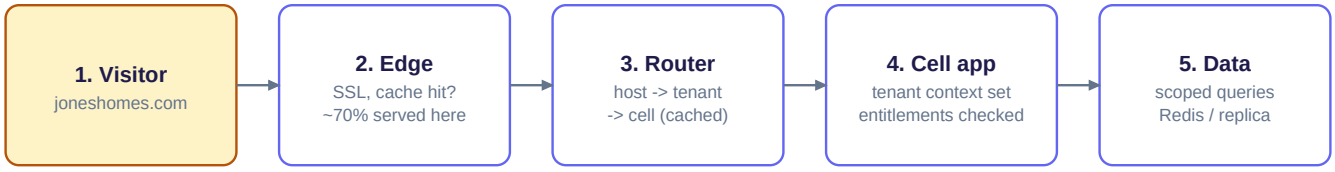


Figure 2 - Components inside one cell

5.2 Request flow



Target: cached page < 100 ms TTFB worldwide; dynamic API p95 < 200 ms

Figure 3 - How a request on a brokerage's own domain is served

Routing data (domain to brokerage to cell) is cached at the edge and in Redis, so the control plane is not on the hot path. If the control plane is down, existing brokerages keep working; only sign-ups and plan changes pause.

6. Multi-tenancy and isolation

Every brokerage is a tenant. Brokerage data lives in its cell's database with a **tenant_id** on every row, and isolation is enforced at every layer rather than trusted to developers.

Layer	How isolation is enforced
Routing	Tenant resolved from the domain or API token before any code runs; requests without a tenant are rejected.
Database	A *BelongsToTenant* trait adds a global scope to every tenant model; composite indexes start with tenant_id; writes fill tenant_id automatically.
Static checks	Architecture tests fail the build if a tenant table lacks the trait or if raw SQL is used outside approved repositories.
Cache, queues	Cache keys and job payloads carry the tenant; jobs restore tenant context before running.
Files	S3 prefix per brokerage, signed URLs only, KMS key per brokerage for sensitive documents.
Search	Every OpenSearch query carries a mandatory tenant filter added by the platform layer.
AI	Retrieval filtered by tenant, role and permissions before the model sees any content (Section 13).
Secrets and PII	SSNs, bank details and tax IDs encrypted per brokerage (envelope encryption); never in logs.
Verification	A cross-tenant test suite on every build creates two brokerages and tries to read across them through every API.

6.1 Hierarchy inside a tenant

Brokerage -> offices -> teams -> agents, with roles at each level. Permissions use RBAC plus record ownership: an agent sees their own contacts and transactions, a team lead sees the team's, an office manager sees the office's, and a broker sees all. Brokerages can create custom roles from a permission catalogue that each module contributes.

6.2 Dedicated cells

An enterprise brokerage can be placed in its own cell (own database, own keys, chosen region) with no code change. Moving a brokerage between cells is an automated, rehearsed operation (copy, verify, switch route, clean up).

6.3 Custom domains

A broker enters *joneshomes.com*; RightAlly shows the CNAME record to add; Cloudflare for SaaS validates it and issues SSL automatically. Agents get sub-domains (*jane.joneshomes.com*) or their own domains on higher packages. The portal (back office) is served on the brokerage domain or on *brokerage.rightally.app*.

7. Scalability and performance

Capacity grows by adding cells. A cell is sized by load (active agents and requests), not by brokerage count; most brokerages are small, so one cell typically holds up to about 2,000 brokerages or 50,000 agents.



Figure 4 - Growth path: the same software, more cells

7.1 Performance targets (SLOs)

Measure	Target	How we meet it
Public IDX / agent pages (cached)	< 100 ms TTFB, global	Edge full-page cache, purged on listing or content change
Logged-in API, p95	< 200 ms	Octane in memory, Redis cache, read replicas, no N+1 queries (enforced)
Listing search, p95	< 300 ms	OpenSearch, pre-computed geo shapes, result caching
Page interactive on 4G mobile	< 2.5 s LCP	SSR, code splitting, image CDN with modern formats
Realtime notification	< 1 s	Reverb websockets
Availability	99.95% per month	Multi-AZ cells, rolling deploys, auto-failover databases

7.2 Scaling levers, in order of use

- **Cache first:** edge cache for public pages, Redis for sessions, settings, entitlements, dashboards and computed KPIs.
- **Do it later:** anything not needed in the response (emails, PDFs, syncs, reports, AI checks) runs on queues.
- **Scale out:** stateless app and worker pods autoscale on CPU and queue depth.
- **Read from replicas:** reports and dashboards read from replicas; analytics from the warehouse, never the primary.
- **Add cells:** new brokerages land in the least-loaded cell; hot cells are split by moving brokerages.
- **Extract a module:** only if one module's load profile justifies its own service (likely candidates: listing search, messaging, AI).

7.3 Proving it

Each phase includes k6 load tests against a production-sized cell (50,000 agents of synthetic data). Before GA we run a multi-cell test at the 1-million-agent level and publish the results with this document's targets.

8. Technology stack

Layer	Technology	Notes
Language / framework	PHP 8.5, Laravel 13	Strict types, PHP attributes, Laravel AI SDK
App server	Laravel Octane (Swoole)	Long-lived workers, concurrent tasks
Front end	React 19, TypeScript, Inertia 2 (SSR), Tailwind CSS 4	Shared design system; Storybook for components
Site rendering	SSR React blocks + edge cache	Same block engine for IDX templates and agent pages
Database	MySQL 8.4 LTS (Aurora)	utf8mb4, InnoDB, FKs, decimal money
Search	OpenSearch	Listings (per MLS), CRM and document search (per cell)
Cache / queues / realtime	Redis, Horizon, Reverb	Separate Redis for cache and queues
Files	S3 + CloudFront / Cloudflare images	Per-brokerage prefixes and keys
Analytics	Event stream to ClickHouse (or Redshift)	Reports across millions of rows without touching OLTP
AI	Laravel AI SDK, LLM gateway, vector index	Provider-agnostic; per-tenant retrieval filters
Payments	Stripe Billing + Stripe Connect	Subscriptions, seats, agent fees, payouts
Messaging	Amazon SES, Twilio (A2P 10DLC), FCM	Email, SMS, push
Auth	Fortify, Sanctum, Passport (partners), WebAuthn	MFA, SSO (Google, Microsoft), API tokens
Packages	spatie permission, activitylog, medialibrary, settings	Proven building blocks
Quality	Pest, PHPStan level 9, Larastan, Pint, Vitest, Playwright	Architecture tests enforce module boundaries
Infra	Docker, Terraform, EKS or ECS, GitHub Actions	Infrastructure as code, per-cell stacks
Observability	OpenTelemetry, Grafana / Datadog, Sentry	Traces, metrics, logs per cell and tenant

9. Module catalogue: what we build

Twenty modules in four suites on one Core. Each module is sold in packages or as an add-on, and each must be best in its category on its own. Shared data (contacts, listings, transactions, documents, agents) flows between them automatically.

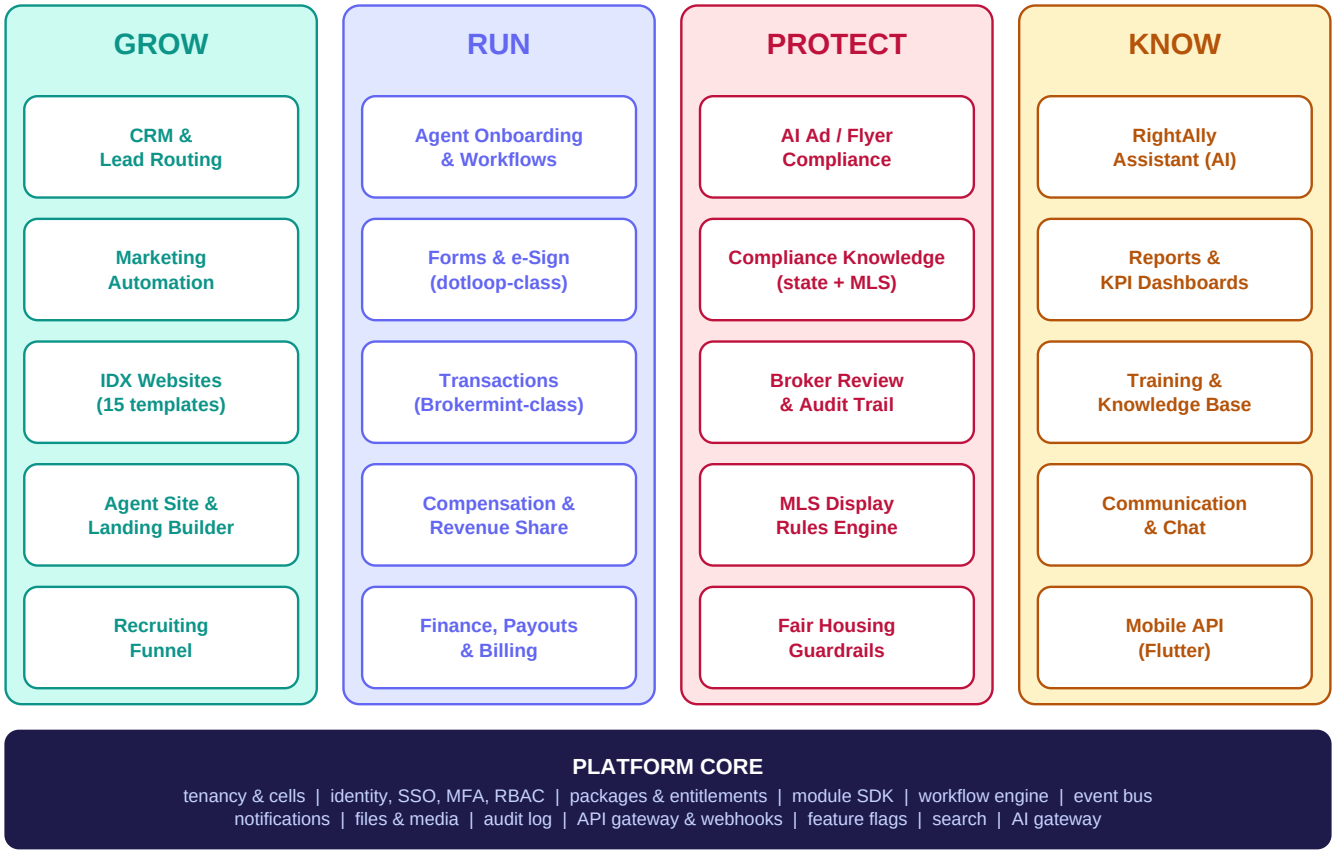


Figure 5 - Module map: four suites on one Platform Core

9.1 Platform Core (always included)

Capability	Features
Tenancy & provisioning	Self-service brokerage signup, cell placement, guided setup wizard, sandbox brokerage for demos
Identity & access	Email / SSO login, MFA, passkeys, sessions per device, RBAC with custom roles, offices and teams, impersonation with audit
Packages & entitlements	Super admin builds packages and add-ons, seat-based pricing, trials, coupons; broker assigns modules and seats to agents
Workflow engine	Versioned workflows: triggers, forms, document upload, addendum, e-sign, payment, approval, wait, branch, notify; reused by every module
Notifications	In-app, email, SMS, push; user preferences; templates per brokerage
Files & media	Upload, virus scan, image optimisation, versioning, retention rules
Audit & activity	Who did what, when, from where, for every record; exportable for compliance
API & webhooks	REST v1 with OpenAPI docs, tokens per integration, webhooks with retries and signatures
Branding & settings	Logo, colours, fonts, email sender domain, legal footers, per-office overrides

9.2 GROW suite

Module	Key features	Benchmark to beat
CRM & Lead Routing	Unified contact timeline (calls, texts, emails, site visits, saved searches); smart lists; pipelines for buyers, sellers, recruits; round-robin, geographic and performance-based lead routing with SLAs and re-routing; tasks and reminders; two-way email/calendar sync; click-to-call and SMS; duplicate merge; lead scoring from website behaviour; team and broker oversight	BoldTrail (kvCORE), Follow Up Boss, Lofty
Marketing Automation	Drip campaigns by behaviour (viewed listing, price drop, anniversary); listing alerts and market reports; email and SMS builder; social post scheduling; A2P 10DLC registration; unsubscribe and consent tracking (CAN-SPAM, TCPA); campaign analytics	BoldTrail Smart Campaigns, Lofty
IDX Websites	15 premium, conversion-tested brokerage templates; map and list search, polygon draw, saved searches, alerts; listing pages with schema.org SEO; neighbourhood and school pages; lead capture with configurable registration gates; agent roster and profiles; blog; MLS rule compliance automatic	BoldTrail sites, Luxury Presence, Real Geeks
Agent Site & Landing Builder	Drag-and-drop section blocks; templates for listing, open house, buyer guide, home valuation, recruiting; agent sub-domain or own domain; every CTA creates a CRM lead for that agent; 'Join' CTAs start the brokerage onboarding workflow; A/B tests; brokerage-locked compliance blocks	Lofty pages, Carrd / Unbounce for real estate
Recruiting Funnel	Recruit pipeline, revenue-share calculator page for prospects, referral tracking from agent pages, onboarding hand-off	Brokerage recruiting tools

9.3 RUN suite (back office)

Module	Key features	Benchmark to beat
Agent Onboarding & Workflows	Today's flow rebuilt as workflow template #1: register -> documents and addendum -> agreement e-sign -> payment -> approval -> welcome; resume later; versioned agreements with acceptance records; license and W-9 capture; MLS and board onboarding checklist	Today's RightAlly flow, SkySlope onboarding
Forms & e-Sign	Transaction rooms (loops) with participants and roles; state form library with auto-fill from transaction and MLS data; fillable field mapping; clause library; e-sign with initials, dates, in-person and remote signing, signing order; ESIGN/UETA audit certificate; templates and task lists; mobile signing; document versioning; AI extraction of key dates and prices	dotloop, Lone Wolf Transact, DocuSign Rooms
Transactions	Listing, buyer, dual, lease and referral transactions; pipeline from pre-contract to closed; checklists by type and state; broker review queue with comments and rejections; key-date tracking (inspection, appraisal, closing); earnest money and escrow tracking; commission disbursement authorization (CDA) generation	Brokermint, SkySlope
Compensation & Revenue Share	Configurable plans: registration fee, monthly and per-agent fees, revenue split, flat fee, caps with reset periods, post-cap fees, rank and target based splits, team splits, sponsor and co-sponsor shares, multi-level revenue share with eligibility and root fallback, deductions catalogue; plan versions with effective dates; what-if simulator for agents and recruits	Today's RightAlly engine (all rules preserved), eXp / Real-style models
Finance, Payouts & Billing	Double-entry ledger for every commission, fee and payout; agent billing (invoices, auto-charge, failed-payment retry); payouts via ACH (Stripe Connect or bank file); 1099-NEC generation; trust and escrow reports; QuickBooks Online and Xero sync; GCI, company dollar, cap status and P&L reports	Brokermint back office, Lone Wolf

9.4 PROTECT suite (compliance)

Module	Key features	Benchmark to beat
AI Ad / Flyer Compliance	Agent uploads flyer, social post, email or landing page; AI checks brokerage name and licence disclosure, Equal Housing logo, fair housing language, MLS and state advertising rules, 'coming soon' and pocket-listing rules; returns issues with the rule cited and suggested fixes; broker approve / reject queue; history per agent	New category - few competitors
Compliance Knowledge	NotebookLM-style library: state real estate law, commission rules, MLS rules, brokerage policy manual; answers with citations to the exact source; brokerage can add its own documents; change alerts when rules are updated	Generic AI tools without real estate sources
Broker Review & Audit	Single review queue across documents, ads and transactions; evidence packs for audits by state regulators	SkySlope audit tools
MLS Display Rules Engine	Per-MLS settings for attribution, disclaimers, logos, sold data, refresh windows, field visibility; applied automatically to every page	Required for IDX approval

9.5 KNOW suite (intelligence and engagement)

Module	Key features
RightAlly Assistant	Internal chatbot for agents and staff: 'how do I...', 'what is my cap status', 'draft a follow-up to this lead'; answers only from the user's brokerage and permitted records; can take actions with confirmation
Reports & KPIs	Role dashboards (today's 40 KPI definitions preserved), leaderboards, goals, custom report builder, scheduled reports
Training & Knowledge Base	Courses, videos, quizzes, policy manual, onboarding tracks, completion tracking
Communication & Chat	Team chat, announcements, inbox, events, meetings (Zoom / Meet links)
Mobile API	Every feature exposed through versioned APIs for the Flutter apps

10. Building a new module (Module SDK)

Every feature, including the ones in Section 9, is built the same way, so adding a module in year three is as safe as adding one in month three. A module is a self-contained package with a manifest, its own tables and its own UI, registered with Core through contracts.



Figure 6 - Anatomy of a module and how it connects to Core

10.1 Steps to add a module

1. Write a one-page module brief and ADR: problem, users, data it owns, events it publishes and consumes.
2. Scaffold with `php artisan module:make Showings`: manifest, folders, provider, test suite.
3. Create migrations for the module's own tables (prefixed, tenant_id included, FKs only within the module or to Core IDs).
4. Register routes, menus, permissions, settings and dashboard widgets through Core contracts.
5. Expose a public interface and domain events for other modules; never read another module's tables.

6. Add the entitlement key; super admin adds the module to packages or as a paid add-on.
7. Ship behind a feature flag; enable for pilot brokerages; then general availability.

10.2 When a module needs its own database or datastore

Default: the module's tables live in the cell's MySQL database, owned exclusively by that module. A separate datastore is allowed when the data shape or load clearly needs it, approved by ADR:

Need	Datastore	Example
Relational, transactional (default)	Cell MySQL, module-prefixed tables	Transactions, CRM, comp plans
Full-text or geo search	OpenSearch index owned by the module	Listings, contact search
High-volume events and analytics	ClickHouse / warehouse tables	Website visits, email opens
Vectors for AI	Vector index with tenant metadata	Compliance library, brokerage docs
Large files	S3 bucket prefix	Signed documents, flyers
Separate scale or team	Module's own database + service	Future extraction of messaging or search

Rules for every datastore: tenant_id on every record, backups and restore tests, encryption at rest, and an owner module.

10.3 Brokerage-specific needs without forks

Brokerage differences are handled in this order: settings, then workflows and templates, then custom fields and forms, then integrations through API and webhooks, and finally a private module published only to that brokerage. Core code is never edited for one customer, so every brokerage receives every release.

11. Data architecture

The new schema is designed from the business domains, not copied from the legacy tables. The legacy analysis (Phase 0) found 146 tables with no foreign keys and money stored as text; none of those patterns carry over.

Standard	Rule
Keys	bigint auto-increment internally; ULID public IDs in URLs and APIs
Integrity	Foreign keys on all relations; restrict deletes on financial records
Money	decimal(15,2) with currency, or integer cents in the ledger; never float or text
Time	UTC datetime columns; brokerage time zone applied in presentation
Audit	created_at, updated_at, created_by, updated_by; soft deletes where users can delete; full activity log
Concurrency	Optimistic locking (version column) on transactions, plans and documents
History	Versioned tables for comp plans, workflows, agreements, deduction rules and MLS rules
Naming	Plural snake_case tables, module prefix, English spelling checked in review
Charset	utf8mb4, InnoDB only
Indexes	tenant_id first in composite indexes; every query plan reviewed for list and report screens

11.1 Migrations without downtime

Schema changes follow **expand, migrate, contract**: add the new column or table (backward compatible), deploy code that writes both, backfill in batches by queue, switch reads, then remove the old structure in a later release. Large-table changes use online schema change tooling. Migrations run once per cell during the rollout, never by brokerages.

11.2 The commission ledger

The best part of the current system, the 2026 ledger design (calculation snapshots, allocations, cap periods, entitlements, double-entry journals, wallet ledger), becomes the foundation of the Finance module. Every money movement is an immutable journal entry with an idempotency key; balances are derived. The 17 business rules catalogued in Phase 0 become automated tests before the engine is accepted.

12. MLS and IDX architecture

Each brokerage provides MLS authorisation when onboarding; RightAlly ingests each MLS once and serves its listings to every authorised brokerage in that MLS. Launch markets: Mississippi and one additional state.

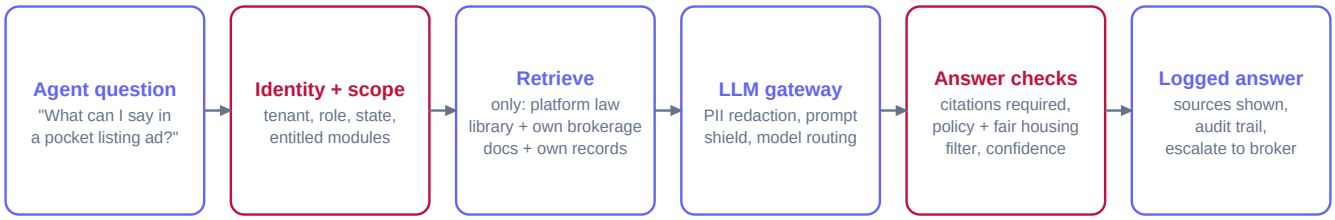
Stage	Design
Connect	RESO Web API (MLS Grid, Bridge, Trestle or direct); credentials stored encrypted per MLS and per brokerage authorisation
Ingest	Initial full load, then incremental sync by modification timestamp every few minutes; deletes and status changes tracked
Normalise	RESO Data Dictionary fields mapped to one listing model; MLS-specific fields kept in an extension block
Media	Photos fetched once to S3, resized and served from the image CDN
Index	OpenSearch index per region with geo shapes, facets, school and neighbourhood joins
Entitle	A brokerage sees an MLS's listings only while its authorisation is active
Display	MLS Display Rules Engine applies attribution, disclaimers, sold-data and field rules on every page
Cache	Listing pages edge-cached and purged on change

12.1 IDX templates

Fifteen premium templates built on one block engine shared with the Agent Site Builder: hero search, featured listings, communities, agent roster, testimonials, valuation, recruiting and blog blocks. Each template is mobile-first, accessible (WCAG 2.2 AA), scores 90+ on Core Web Vitals, and is A/B tested for lead conversion. Brokerage branding is applied by design tokens, so a template change reaches every brokerage using it.

13. AI architecture and guardrails

Three AI products share one AI platform: Ad / Flyer Compliance, Compliance Knowledge and the RightAlly Assistant. All AI calls go through one gateway that enforces tenant scope, redacts personal data, logs usage and lets us change model providers without code changes.



Hard rule: retrieval filters are applied by the platform before the model sees anything - the model is never trusted to keep tenants apart.

Figure 7 - Guarded AI request flow

Product	How it works	Guardrails
Ad / Flyer Compliance	Image and text extraction from the upload; checks against a rules catalogue per state and MLS plus brokerage policy; LLM explains each finding	Findings cite the rule; final decision always by the broker; no promise of guaranteed compliance
Compliance Knowledge	Curated library of state law, MLS rules and brokerage manuals, chunked and embedded with source and effective date	Answers must cite sources; 'I don't know' when not found; library updates reviewed by RightAlly
RightAlly Assistant	Retrieves from help docs and the user's permitted records; can propose actions	Tenant and permission filters before retrieval; actions require user confirmation; no cross-brokerage data

Positioning. AI compliance features flag issues and cite rules; they do not certify compliance. The broker remains the approver, and every check is logged. This protects RightAlly from liability and matches how brokerages are regulated.

14. Releasing to all brokerages

Brokerages never install updates. Every release is deployed centrally to every cell through the same pipeline, with automatic checks at each step. This replaces the current per-brokerage git pull.

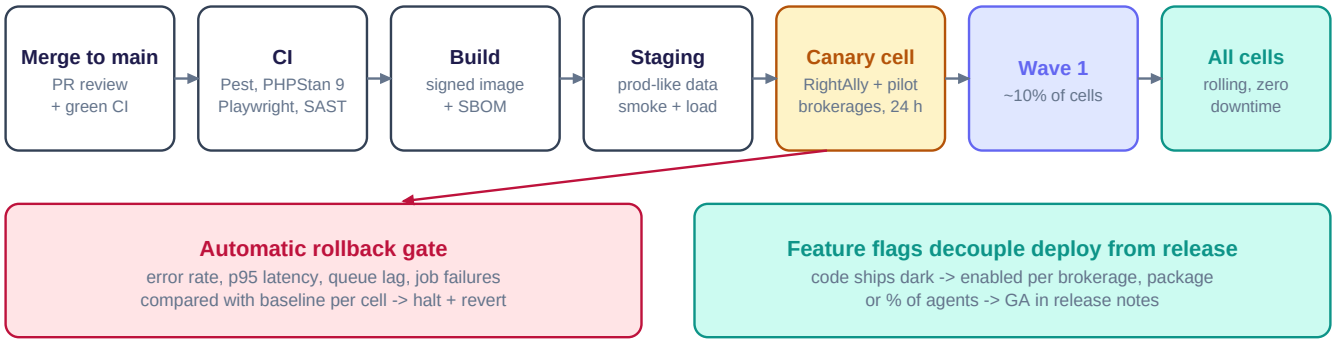


Figure 8 - Release pipeline from merge to every cell

Topic	How it works
Cadence	Small releases several times a week; security fixes same day through the same pipeline
Deploy vs release	Code is deployed dark; features are released by flag to pilot brokerages, then by package, then to all
Database changes	Expand-migrate-contract, so old and new code both work during a rolling deploy
Rollback	Previous image redeployed automatically if health gates fail; data changes are backward compatible
New modules	Deployed to all cells but visible only to brokerages entitled to them (package or add-on)
API compatibility	API v1 never breaks; breaking changes go to v2 with a deprecation window of at least 12 months
Customisations	Settings, workflows, templates, custom fields and private modules are data or packages outside Core, so they survive every release
Communication	In-app release notes, 'what's new' tours, admin email for changes needing action
Enterprise control	Dedicated-cell brokerages can choose a release window within 14 days

15. Security and compliance

Area	Controls
Application	OWASP ASVS level 2; CSRF, XSS and SQL injection protection by framework defaults; CSP; rate limiting; signed webhooks; dependency and container scanning in CI
Identity	MFA (required for admins), passkeys, SSO, session management, least-privilege roles, quarterly access reviews
Data	Encryption in transit (TLS 1.2+) and at rest (KMS); per-brokerage keys for sensitive fields and documents; PII never in logs
Secrets	AWS Secrets Manager; no secrets in code or repositories (Phase 0 found live keys in source; this is closed by design)
Audit	Immutable activity log for users, admins and RightAlly support access
Certification	SOC 2 Type II readiness from day one; audit targeted in year two
Real estate rules	MLS IDX rules, fair housing advertising checks, state license disclosure, ESIGN / UETA for e-signatures, record retention by state
Marketing rules	CAN-SPAM, TCPA consent records, A2P 10DLC registration, unsubscribe handling
Privacy	US state privacy laws (e.g. CCPA/CPRA): data export and deletion requests, data processing agreements with brokerages
Payments	Stripe handles card data (PCI DSS SAQ A); bank details tokenised

16. Reliability, observability and disaster recovery

Topic	Design
Availability	Each cell spans three availability zones; database failover is automatic
Backups	Continuous point-in-time recovery (35 days) plus daily snapshots copied to another region
RPO / RTO	Recovery point under 5 minutes; recovery time under 1 hour per cell; whole-region recovery under 4 hours
Restore tests	Automated monthly restore of a random cell into an isolated environment, with data checks
Brokerage restore	Restore one brokerage's data to a point in time without affecting others
Monitoring	Golden signals per cell and per brokerage; queue lag; MLS sync freshness; payment and payout health
Alerting	On-call rota, runbooks for every alert, public status page
Tracing	OpenTelemetry traces from edge to database for every request

17. Roadmap and team

About 8-9 months to a sellable MVP and about 13 months to the full suite, with a core team of eleven. Dates are estimates and will be firmed up at the end of Phase 1.

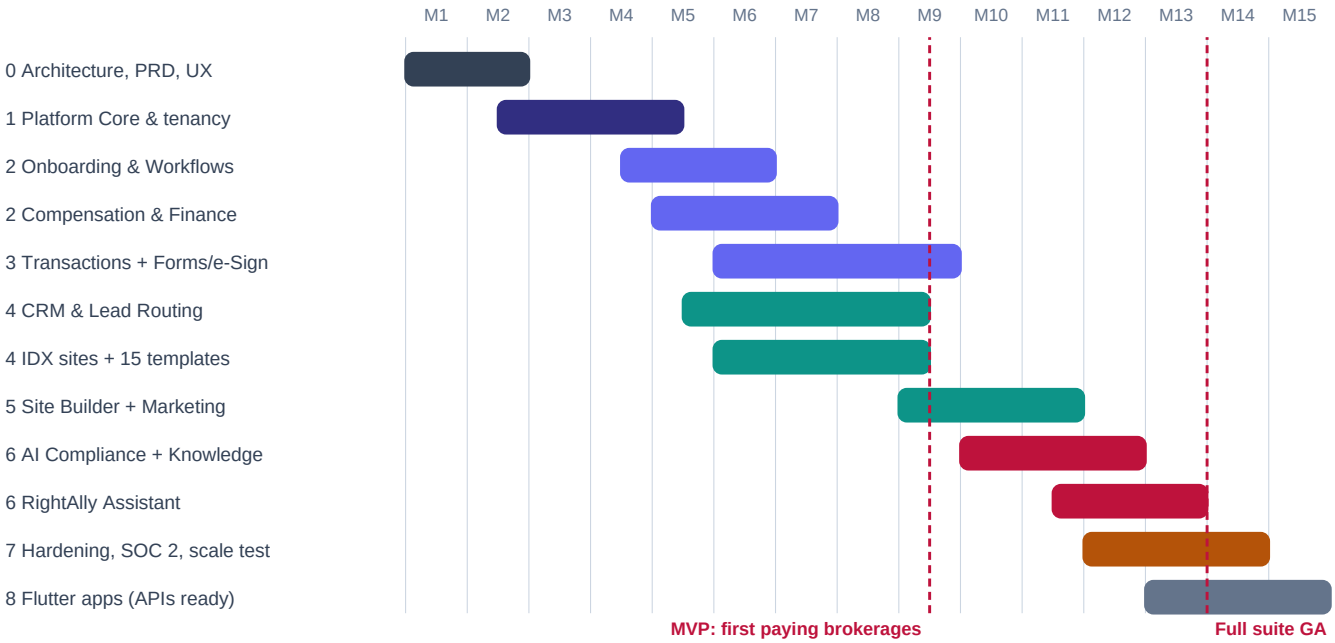


Figure 9 - Indicative 15-month roadmap (months from approval)

Role	Count	From
Architect / tech lead	1	Month 1
Laravel engineers	4	Month 2
React engineers	3	Month 2
UI/UX designer	1	Month 1
QA automation engineer	1	Month 2
DevOps / SRE engineer	1	Month 2
AI engineer	1	Month 8
Flutter engineers	2	Month 12
Product owner (RightAlly)	1	Month 1

Delivery runs in short sprints. Each sprint starts with a detailed specification, must pass CI fully and is merged before the next sprint starts.

18. Risks and mitigations

Risk	Impact	Mitigation
MLS approvals take months per MLS	IDX launch delayed	Start vendor applications now for Mississippi and the second state; build against RESO test feeds
State association forms are licensed	Forms module cannot ship official forms	Negotiate form licences with state associations; launch with brokerage-owned templates
Scope: 20 modules, each 'the best'	Late or shallow product	MVP focuses on back office + IDX + CRM core; depth over breadth per release
Commission maths errors	Lost trust, financial liability	Rules as tests, parallel-run against legacy calculations, ledger reconciliation reports
Cross-tenant data leak	Severe	Isolation at every layer, automated leak tests, external penetration test before GA
AI gives wrong compliance advice	Legal exposure	Citations required, broker approval, clear positioning, logged answers
SMS / email deliverability	Marketing fails	Dedicated sending domains per brokerage, 10DLC registration, reputation monitoring
Cloud cost growth	Margin pressure	Per-cell cost dashboards, autoscaling floors, reserved capacity at scale
Team ramp-up on new stack	Slower start	Reference module built first by the tech lead; code review and architecture tests

19. Decisions needed and assumptions

Decisions needed to start Phase 1

#	Decision	Recommendation
1	Approve cell-based shared SaaS (supersedes per-customer deployment in specs 02-04)	Approve
2	Cloud provider	AWS + Cloudflare
3	MLS feed terms: one vendor feed per MLS shared across authorised brokerages, and the feed provider	Confirm with each MLS
4	E-signature: build native (needed for a dotloop-class Forms module) with optional DocuSign integration	Build native
5	State association form licensing for Mississippi and the second state	Start talks now
6	Product owner on the RightAlly side, and GitHub repository name	Name both
7	SOC 2 Type II as a year-two goal	Approve

Assumptions

- Existing brokerage clients are not migrated now; the new platform starts with clean data.
- Legacy MLM features (binary, e-pin, PV, ranks, auto-pool) are dropped; rank and target based splits are rebuilt inside the Compensation module.
- The current onboarding flow (register, documents and addendum, agreement, payment) is the first workflow template.
- Mobile apps are built later in Flutter on the same API.
- Timelines assume the team in Section 17 and timely decisions on the items above.

Next step. On approval, Phase 1 produces the Product Requirements Document module by module, with a detailed specification for each, starting with Platform Core and the Compensation & Finance modules.